STOREFILLER

Team Execution Handbook

4-Week Sprint Task Breakdown · Backend & Mobile · 3 Backend Devs + 1 Mobile Dev

BE-1	BE-2	BE-3	MOB-1
• Auth + Firebase • Shop Onboarding • Notifications • Support + Referral • Admin Ops • CI/CD + Infra	• Catalog + Category • pg_trgm Search • Cart Module • Delivery + Tracking	• Order FSM • Payments • Wallet • Reconciliation	• Buyer Screens • Agent Screens • Admin Screens • React Native Expo — all roles, one app

Task ID Prefix Key: B1/B2/B3 = Backend devs · M1 = Mobile dev. All API paths reference **https://api.storefiller.in/api/v1/...** proxied via Railway. 49 tasks across 4 weeks — use this handbook alongside the Management Roadmap's Gantt tracker and Definition of Done.

WEEK 1 · Jul 27–31, 2026 — FOUNDATION & AUTHENTICATION

Focus: Repo + CI/CD · Database schema (22 models) · Firebase auth · Shop onboarding · Establishes the non-negotiable infrastructure every later module depends on. By Friday, every developer has a working local environment, the full schema deployed to Neon, and Firebase auth tested end-to-end.

CRITICAL PATH

DB schema must be 100% complete by EOD Day 2 (Tue). All 3 BE devs depend on it for Week 2.

CONTRACT FREEZE

Route contract committed by EOD Day 3 (Wed). MOB-1 uses mock data matching it from Day 4.

SECURITY GATE

RS256 key pair generation is BE-1's first task. Private key must never touch the codebase.

ID	ASSIGNEE	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
B1-01	**BE-1**	**Repo & Monorepo Setup**	• Create GitHub monorepo: apps/api, apps/mobile, packages/types • Configure ESLint + Prettier across all workspaces • Set up Husky pre-commit hooks (lint + typecheck) • Initialize Node.js 20 + Express 5 skeleton in apps/api • Configure Pino logger with log levels per environment	✓ Monorepo compiles with no errors ✓ git push triggers ESLint without failures ✓ API server boots on localhost:3000 ✓ GET /api/health returns { status: 'ok', version: '1.0.0' }
B1-02	**BE-1**	**CI/CD Pipeline (GitHub Actions)**	• Create .github/workflows/ci.yml: lint, test, build jobs • Configure Railway CLI in workflow (railway up --service api) • Set GitHub Secrets: RAILWAY_TOKEN, DATABASE_URL, JWT_PRIVATE_KEY, FIREBASE_SERVICE_ACCOUNT • Add smoke test step: curl /api/health after deploy • Set Railway replicas:1 explicitly — prevents duplicate node-cron firing	✓ Pipeline runs green on push to main ✓ Railway deployment succeeds with health check ✓ All secrets set without being exposed in logs ✓ Pipeline fails correctly if lint or build fails
B1-03	**BE-1**	**Neon DB Setup + Extensions**	• Create Neon project, configure pooled connection string • Run CREATE EXTENSION IF NOT EXISTS pg_trgm; • Run CREATE EXTENSION IF NOT EXISTS postgis; • Verify: SELECT extname FROM pg_extension WHERE extname IN ('pg_trgm','postgis') • Set up Prisma: npx prisma init, configure datasource	✓ Neon project active, pooled endpoint noted ✓ pg_trgm and postgis both appear in pg_extension query ✓ Prisma connects to Neon without errors ✓ DATABASE_URL in env vars, not in codebase
B1-04	**BE-1**	**DB Schema — Part 1 (Auth & Shop)**	• Create Prisma models: User, ShopProfile, AgentProfile, DeviceToken • Add Role enum (BUYER/AGENT/ADMIN) • Add indexes: User(phone), DeviceToken(userId) • Run npx prisma migrate dev --name init_auth_shop • Validate with SELECT table_name FROM information_schema.tables	✓ 4 tables created in Neon DB ✓ All FK constraints enforced ✓ Prisma client generated without type errors ✓ Migration file committed to repo

ID	ASSIGNEE	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
B1-05	**BE-1**	**Firebase Admin SDK + Auth Module**	• Install firebase-admin; initialize with service account from Railway env • POST /auth/token/verify: verifyIdToken(), upsert User, issue RS256 JWT + refresh token • Generate RS256 key pair (openssl); store private key in Railway env ONLY • POST /auth/refresh: validate token family, issue new pair, revoke old • Add role guard middleware (BUYER/AGENT/ADMIN)	✓ POST /auth/token/verify returns { accessToken, refreshToken, user } ✓ Invalid Firebase token returns 401 ✓ Refresh token reuse outside grace window invalidates the family ✓ Role guard blocks BUYER from admin routes (403)
B1-06	**BE-1**	**Email OTP Fallback Module**	• Install bcrypt, express-rate-limit • POST /auth/email-otp/send: rate-limit 5/hr, bcrypt-hash OTP, send via Resend • POST /auth/email-otp/verify: validate hash, mark used, issue same JWT shape as Firebase path • node-cron: delete expired OTP rows every hour	✓ POST /auth/email-otp/send returns 200 ✓ OTP stored as bcrypt hash, never plaintext ✓ 6th request within 1hr returns 429 ✓ Fallback path issues an identical session JWT to the Firebase path
B1-07	**BE-1**	**Shop Profile + Cloudinary Signature**	• POST/PATCH /shop/profile with lat/lng • App layer writes float lat/lng AND geography(Point,4326) in the same transaction • POST /shop/photo/upload-signature: return signed Cloudinary params • PATCH /shop/photo/confirm after client-side upload completes	✓ Shop profile created with geocoded location ✓ Signature endpoint returns valid signed upload params ✓ shopPhotoUrl stored after confirm ✓ No image bytes ever appear in API server logs
B2-01	**BE-2**	**DB Schema — Part 2 (Catalog)**	• Create Prisma models: Category (self-relation), Product, ProductInventory • Add ProductUnit enum and version column on Product (optimistic lock) • Raw SQL migration: CREATE INDEX products_name_trgm ... USING GIN (name gin_trgm_ops) • Validate with \d+ "Product" in psql	✓ 3 tables created, GIN trigram index confirmed via \d+ Product ✓ version column present on Product ✓ Migration file committed to repo
B2-02	**BE-2**	**Cloudinary Product Image Preset**	• Create Cloudinary unsigned upload preset restricted to products/ folder • Reserve a separate delivery_photos/ preset for later use • Register webhook route skeleton: POST /products/image-confirm (wired in Week 2) • Define thumbnail transform pattern: w_400,h_300,c_fill,f_webp	✓ Both presets visible in Cloudinary dashboard ✓ Signed test upload from Postman succeeds ✓ Thumbnail URL returns a resized WebP image
B2-03	**BE-2**	**Env Config + Health Monitoring**	• Configure all Railway env vars per the checklist • Set up UptimeRobot monitor on GET /api/health • Extend /api/health to report DB connection + uptime	✓ /api/health returns { db: 'ok', uptime: N } ✓ UptimeRobot monitor shows green status

ID	ASSIGNEE	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
B3-01	**BE-3**	**DB Schema — Part 3 (Orders & Payments)**	• Create Prisma models: Order, OrderItem, Payment, BuyerWallet, WalletTransaction • Add OrderStatus, PaymentMethod, PaymentStatus, WalletTxnType enums • UNIQUE constraint on Payment.razorpayPaymentId (idempotency key) • Add version column to Order for FSM guard	✓ Order table with full status enum exists ✓ UNIQUE constraint on razorpayPaymentId confirmed ✓ All FK chains from Order → Payment validated
B3-02	**BE-3**	**DB Schema — Part 4 (Tracking & Ops)**	• Create remaining 9 models: LiveTracking, DeliveryLocationLog, Settlement, OrderRating, SupportTicket, ReferralCode, Referral, JobRecovery, AuditLog • Raw SQL: add geography(Point,4326) to ShopProfile + LiveTracking, GIST index • AuditLog: grant PG role NO UPDATE/DELETE (append-only enforcement)	✓ All 9 tables created ✓ GIST spatial index confirmed on ShopProfile.location ✓ audit_logs: INSERT works, UPDATE returns permission denied
B3-03	**BE-3**	**Razorpay Account + SDK Skeleton**	• Register Razorpay test-mode account, obtain key ID + secret • Install razorpay npm package, initialize client • Store RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET in Railway env only • Stub POST /payments/razorpay/order (wired fully in Week 3)	✓ Razorpay test dashboard active ✓ SDK initializes without error ✓ Secrets confirmed absent from any committed file
M1-01	**MOB-1**	**Expo Project Setup**	• npx create-expo-app storefiller-mobile --template blank-typescript • Install expo-router, expo-secure-store, expo-location, expo-image-picker • Configure expo-router groups: (auth)/, (buyer)/, (agent)/, (admin)/ • Create AuthContext with Zustand; configure axios client pointing to dev Railway URL	✓ App runs on Android device via Expo Go ✓ expo-router navigation between groups works ✓ expo-secure-store saves and retrieves token correctly
M1-02	**MOB-1**	**Phone Auth + OTP Screens**	• Integrate Firebase phone auth (native module); obtain ID token client-side • Build PhoneInputScreen + OTPInputScreen (auto-focus, 60s resend timer) • On verify: POST idToken to /auth/token/verify, store JWT in expo-secure-store • Route to email-OTP fallback screen if Firebase is unavailable	✓ Phone → OTP → JWT stored flow works on a real device ✓ Email-OTP fallback screen reachable and functional ✓ Token persists across app restart
M1-03	**MOB-1**	**Shop Onboarding Screens**	• Build “Wholesale products for your store” welcome screen • Build shop details form (shop name, owner name, address) with device GPS capture • Wire form submit to POST /shop/profile • Build “All Set!” confirmation screen	✓ Onboarding form submits and creates a ShopProfile ✓ Device GPS permission requested, lat/lng captured ✓ Navigation to home screen after “All Set”

ID	ASSIGNEE	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
M1-04	**MOB-1**	**Shop Photo Upload**	• Integrate expo-image-picker for shop photo capture/selection • Request signed params from /shop/photo/upload-signature • Upload directly to Cloudinary from device (no API server traffic) • Call PATCH /shop/photo/confirm after upload completes	✓ Photo uploads directly to Cloudinary — verified: no multipart traffic in API logs ✓ Photo appears on profile screen after upload ✓ Upload progress indicator shown during transfer

WEEK 2 · Aug 3–7, 2026 — CATALOG, SEARCH & CART

Focus: Category/Product CRUD · pg_trgm search · Cart · Wallet · Mobile browse + cart screens · Buyers can browse, search, and build a cart against the real catalog. The pg_trgm GIN index must be validated with EXPLAIN ANALYZE — not assumed.

INTEGRATION RISK

MOB-1 switches from mock data to real API this week. Any BE endpoint change after Monday needs a PM note + version bump.

SEARCH VALIDATION

BE-2 must show EXPLAIN ANALYZE at Wed sync. GIN index scan must appear — a sequential scan is a blocker.

ID	ASSIGNEE	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
B1-08	**BE-1**	**Admin Product & Category CRUD**	• POST /admin/products with version-based optimistic lock • PATCH /admin/products/:id: version check → 409 on conflict • POST /admin/categories (supports parentId for subcategories) • PATCH /admin/products/:id/inventory (quantityAvailable, version)	✓ Concurrent PATCH with same version → second returns 409 ✓ Category tree returns nested subcategories correctly ✓ Inventory update reflected in ProductInventory table
B1-09	**BE-1**	**Notifications Module (FCM)**	• Install Firebase Cloud Messaging on the same Firebase project as Auth • NotificationService.sendPush(userId, title, body, data) • POST /users/me/device-token; markInactive() on failed send • PATCH /users/me/notification-preferences	✓ FCM push received on a test Android device ✓ Invalid FCM token marks device_tokens.isActive = false ✓ Preference toggle respected before sending a push
B1-10	**BE-1**	**Support & Referral Endpoints**	• POST /support/tickets, GET /support/tickets • GET /referral/code (auto-generate on first request) • GET /referral/history • Stub referral reward logic (credited on referred buyer's first order — wired in Week 3)	✓ Ticket creation returns 201 with the ticket record ✓ Referral code is unique and idempotently generated ✓ Referral history lists referred users with status
B2-04	**BE-2**	**Search Module — pg_trgm + Filters**	• GET /products?q=&categoryId;=&page;=&limit;= • Query: WHERE similarity(name, $q) > 0.25 OR name ILIKE '%'\|\|$q||'%' • Category filter + pagination; return { data, total, page } • Run EXPLAIN ANALYZE — confirm Bitmap Index Scan on the GIN index	✓ GET /products?q=atta returns relevant results <200ms ✓ EXPLAIN ANALYZE shows GIN index scan, not a sequential scan ✓ Empty query returns all active products, paginated
B2-05	**BE-2**	**Cart Module**	• GET /cart (join CartItem + Product for live prices) • POST /cart/items (upsert on buyerId+productId unique constraint) • PATCH /cart/items/:productId, DELETE /cart/items/:productId, DELETE /cart	✓ Cart persists server-side across app restart ✓ Adding an existing product increments quantity, no duplicate row ✓ Cart total recomputed from live product prices, never a stale cache

ID	ASSIGNEE	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
B2-06	**BE-2**	**Cloudinary Product Image Webhook**	• Wire up POST /products/image-confirm webhook • On webhook: update Product.imageUrl • node-cron every 5min: flag ACTIVE products with no image for admin review	✓ Webhook fires and Product.imageUrl set correctly ✓ Thumbnail URL (w_400) returns a resized WebP image ✓ Missing-image products auto-detected by the cron
B3-04	**BE-3**	**Wallet Module**	• GET /wallet (auto-create BuyerWallet at balance 0 on first access) • GET /wallet/transactions • POST /wallet/topup/razorpay-order (creates a Razorpay order for the top-up amount)	✓ New buyer's wallet auto-initializes at balance 0 ✓ Top-up creates a valid Razorpay order ✓ Transaction history shows CREDIT/DEBIT entries with a running balance
B3-05	**BE-3**	**Razorpay Order Creation (checkout prep)**	• POST /payments/razorpay/order: create order via razorpay.orders.create() • Store razorpayOrderId against a PENDING Payment record • Return { razorpayOrderId, amount, currency, keyId } to the client	✓ Razorpay test-mode order created successfully ✓ Payment row inserted in PENDING state ✓ Amount matches cart subtotal + fees exactly, in paise
M1-05	**MOB-1**	**Home + Category Browse Screens**	• Build HomeScreen: category chips, featured products grid • Build CategoryScreen: nested subcategory navigation (e.g. Grocery → Atta, Rice...) • Wire to GET /categories and GET /products	✓ HomeScreen renders categories from the real API ✓ Category navigation drills into subcategories correctly ✓ Pull-to-refresh updates the product grid
M1-06	**MOB-1**	**Product Detail + Search**	• Build ProductDetailScreen: image, price, unit/case info, quantity selector, add-to-cart • Build a debounced (300ms) search bar hitting GET /products?q= • Skeleton loading states for slow connections	✓ Search returns real results from pg_trgm with no perceptible lag ✓ Product detail shows case-pack info (e.g. “Sold in boxes of 12”) ✓ Skeleton loaders shown during fetch
M1-07	**MOB-1**	**Cart Screen**	• Build CartScreen: line items, quantity steppers, remove item, running total • Wire to GET/POST/PATCH/DELETE /cart endpoints • Empty-cart state with a “browse products” CTA	✓ Cart screen reflects server state, not local-only storage ✓ Quantity changes debounce-sync to the API without UI lag ✓ Empty state renders cleanly
M1-08	**MOB-1**	**Admin Catalog Screens**	• Build AdminProductsScreen: list, create/edit form, inventory editor • Build AdminCategoriesScreen: create category, assign parent • Handle 409 optimistic-lock conflict with a “reload and retry” toast	✓ Admin can create/edit a product from the mobile admin tab ✓ 409 conflict shows a clear retry prompt, never a silent failure ✓ New category appears in buyer-side category list immediately

WEEK 3 · Aug 10–14, 2026 — ORDERS, PAYMENTS & DELIVERY

Focus: Order FSM · Inventory locking · Razorpay + Wallet + COD · Geofenced delivery · Receipts · The most technically demanding week. The order state machine and payment infrastructure are the financial heart of the platform and have zero tolerance for correctness errors.

NON-NEGOTIABLE #1

Inventory lock uses SELECT FOR UPDATE — never an application-level check.

NON-NEGOTIABLE #2

Webhook HMAC verified against the raw body; express.raw() must load before express.json().

NON-NEGOTIABLE #3

Duplicate webhook idempotency enforced by a DB UNIQUE constraint, not application logic.

Mandatory tests before Friday demo: (1) 10 concurrent POST /orders on 1-unit stock → exactly 1 success, 9×409 (2) duplicate webhook sent 3× → payments table has exactly 1 row (3) invalid state skip → 422 (4) geofence >200m → 422, ≤200m → DELIVERED (5) server killed mid-payment → restart → JobRecovery re-processes it.

ID	ASSIGNEE	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
B1-11	**BE-1**	**Admin Order Operations**	• GET /admin/orders?status= • PATCH /admin/orders/:id/confirm (manual COD confirmation) • PATCH /admin/orders/:id/assign-agent (checks agentProfile.isAvailable) • GET /admin/dashboard (pending orders, pending settlements, low-stock counts)	✓ Admin order queue filters correctly by status ✓ Assigning an unavailable agent returns 400 with a clear message ✓ Dashboard counts match live DB state
B1-12	**BE-1**	**Audit Log Wiring**	• Middleware: write an AuditLog row on every admin mutation (product edit, order assign, settlement pay) • GET /admin/audit-logs with filters by action/date	✓ Every admin action produces exactly one AuditLog row ✓ Audit log viewer shows action, entity, admin, timestamp in order
B2-07	**BE-2**	**Delivery Tracking Endpoints**	• POST /agent/orders/:id/location: upsert LiveTracking + insert DeliveryLocationLog • GET /orders/:id/tracking: read from LiveTracking (O(1), not the log table) • Test with Postman: 50 rapid location posts → tracking always returns latest	✓ Location row updates every 15s from the agent app ✓ GET /tracking returns latest lat/lng within expected latency ✓ History log accumulates without slowing the live-read path
B2-08	**BE-2**	**Geofence Validation + Cleanup Cron**	• PATCH /agent/orders/:id/deliver: run ST_Distance(agent point, shop.location) • ≤200m → transition to DELIVERED; >200m → 422 “too far from shop” • node-cron: DELETE DeliveryLocationLog WHERE recordedAt < NOW() - 30 days	✓ 150m test delivery → 200 + DELIVERED ✓ 250m test delivery → 422, order stays OUT_FOR_DELIVERY ✓ Cleanup cron removes rows older than 30 days on manual trigger

ID	ASSIGNEE	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
B3-06	**BE-3**	**Order FSM + Inventory Lock (core)**	• assertTransition(current, next) map + 422 on any invalid transition • POST /orders: TX → SELECT ProductInventory FOR UPDATE → check availability → increment quantityReserved → INSERT Order + OrderItems (price/name snapshot) → COMMIT • Server recomputes total from live cart + catalog, ignores any client-sent amount • Write concurrency test: 10 simultaneous POST /orders for 1-unit stock	✓ assertTransition rejects PENDING_PAYMENT → DELIVERED skip with 422 ✓ Concurrency test: exactly 1 of 10 simultaneous orders succeeds ✓ quantityReserved increments correctly under load
B3-07	**BE-3**	**Razorpay Webhook + Wallet Debit + COD**	• POST /payments/razorpay/webhook: express.raw() before express.json(), verify HMAC • INSERT Payment ON CONFLICT (razorpayPaymentId) DO NOTHING (idempotency) • Wallet payment: debit BuyerWallet.balance atomically in the same TX as order creation; reject if insufficient • COD orders skip the payment step entirely, go straight to CONFIRMED	✓ Duplicate webhook: second call returns 200, payments table has 1 row not 2 ✓ Wallet debit fails cleanly with 402 on insufficient balance ✓ COD order created directly as CONFIRMED, paymentStatus NOT_APPLICABLE
B3-08	**BE-3**	**Reconciliation Cron + PDF Receipts**	• ReconciliationJob: every 5min, query PENDING_PAYMENT orders >15min old • Call razorpay.orders.fetchPayments() for each; process as webhook if captured • Install pdfkit; GET /orders/:id/receipt streams a PDF generated on request — nothing written to storage • Write a JobRecovery row before every state transition that triggers a downstream action	✓ ReconciliationJob detects an orphaned order and processes the payment ✓ Receipt PDF opens correctly when streamed to the app ✓ Killing the server mid-payment and restarting re-processes the pending JobRecovery row
M1-09	**MOB-1**	**Checkout + Payment Screens**	• Build CheckoutScreen: read-only shop address, payment method selector (Wallet/UPI/Card/COD) • Integrate Razorpay React Native SDK for the UPI/Card path • Handle payment success/failure callbacks • Wallet path: show balance client-side, but treat server as the source of truth	✓ Razorpay checkout opens with the correct amount ✓ Test payment completes and order status updates to CONFIRMED ✓ COD order places instantly with no payment SDK involved
M1-10	**MOB-1**	**Order List, Detail & Receipt**	• Build OrdersScreen: list with status badges • Build OrderDetailScreen: item list, status stepper (Confirmed → Dispatched → Out for Delivery → Delivered) • Receipt view/download button hitting GET /orders/:id/receipt	✓ Order status stepper reflects the correct active step ✓ Receipt opens/downloads correctly from order detail ✓ Pull-to-refresh updates order status live
M1-11	**MOB-1**	**Buyer Tracking Screen**	• Build TrackingScreen: map view with agent marker, 15s polling via GET /orders/:id/tracking • Order status badge above the map; stop polling on DELIVERED • Rating prompt (stars + comment) shown after delivery confirmation	✓ Agent marker updates on the map every 15s during active delivery ✓ Polling stops automatically once status = DELIVERED ✓ Rating submits successfully to POST /orders/:id/rating

ID	ASSIGNEE	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
M1-12	**MOB-1**	**Agent Delivery Screens**	• Build AgentHomeScreen: assigned orders list • Build ActiveDeliveryScreen: “Start Delivery” button, background location task posting every 15s • “Mark Delivered” button → PATCH /agent/orders/:id/deliver with current GPS • Handle geofence rejection with a clear “too far from shop” alert	✓ Agent location POSTed every 15s during an active delivery (verified in DB) ✓ Geofence rejection shows a clear in-app error, never a silent failure ✓ Order transitions to DELIVERED after a successful in-range confirmation

WEEK 4 · Aug 17–21, 2026 — TESTING, HARDENING & RELEASE

Focus: E2E testing · Bug bash · Security hardening · Production deploy · APK build · No new feature development — only bug fixes, hardening, and deployment. Friday of Week 4 is a hard release gate: all 10 Definition of Done criteria (Management Roadmap § 1.4) must be met.

RELEASE GATE (Fri Aug 21)

Production URL live. APK installed. Load test passed. All 10 DoD criteria met. PM sign-off obtained.

BUG PRIORITY

P0 (same day): payment/order integrity, auth bypass, 500s on critical paths. P1: broken UI flows. P2: Phase 2 backlog.

SCOPE CUT AUTHORITY

PM can cut to Phase 2 without a team vote: analytics, bulk admin ops, non-critical polish.

ID	ASSIGNEE	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
B1-13	**BE-1**	**Security Hardening**	• Verify Helmet.js headers active (X-Content-Type, X-Frame-Options, CSP) • Pino redact config: phone numbers and tokens never appear in logs • Confirm JWT private key absent from git history (git log --all | grep PRIVATE_KEY) • express-rate-limit enforced on OTP + auth routes	✓ Security headers present in every API response ✓ No sensitive data in logs ✓ Private key confirmed absent from all git history
B1-14	**BE-1**	**Admin Panel Final Features + Production Deploy**	• Complete settlement pay flow (PATCH /admin/settlements/:id/pay) • Final audit log viewer polish • Merge all branches to main, run full CI, deploy to Railway production • Confirm Railway replicas:1, verify UptimeRobot green	✓ Settlement completion updates status in DB ✓ Railway production deploy successful ✓ UptimeRobot shows all monitors green
B2-09	**BE-2**	**End-to-End Integration Testing**	• Install jest + supertest, configure a test DB (Neon branch) • Happy-path test: auth → browse → cart → order → payment → delivery • Error-path tests: invalid transitions (422), duplicate webhook (200 idempotent), concurrent inventory (409) • Target >70% coverage on catalog, cart, and delivery modules	✓ Happy path integration test passes end-to-end ✓ Concurrent inventory test: exactly 1 success, 9 failures ✓ Coverage report >70% on critical modules
B2-10	**BE-2**	**Cron Audit + Cloudinary Check**	• List all node-cron jobs, verify no overlapping schedules • Stagger schedules, confirm IST timezone on all crons • Check Cloudinary bandwidth dashboard against the 25GB budget	✓ No cron schedule overlaps within a 1-hour window ✓ All crons confirmed running in IST ✓ Cloudinary usage tracked and within budget
B3-09	**BE-3**	**Payment & Order E2E Tests + Load Test**	• Jest/Supertest tests for the full FSM + payment idempotency • k6 or autocannon: 30 concurrent users, 5 min duration against Railway • Monitor Railway CPU/memory during the load test • Confirm reconciliation cron catches a manually-orphaned test order	✓ Duplicate webhook test: 3 calls → 1 payments row ✓ p95 API response time <500ms under 30 concurrent users ✓ No OOM kills during the load test
B3-10	**BE-3**	**Free-Tier Monitoring Runbook + Handoff**	• Document all free-tier thresholds and alerting rules in a shared doc • Set Neon storage alert and Cloudinary bandwidth alert • Document the manual settlement SOP for admin	✓ Monitoring runbook published and shared ✓ Alerts configured and tested ✓ Settlement SOP written and tested by the PM

ID	ASSIGNEE	MODULE / TASK	ACTIONABLE STEPS	EXPECTED OUTPUT
M1-13	**MOB-1**	**Full Bug Bash — All 3 Roles**	• End-to-end test on a physical Android device: onboarding → browse → order → delivery → rating, for Buyer, Agent, and Admin • Fix all critical bugs found during Week 3 testing • Final UI consistency pass (spacing, colors, typography)	✓ All 3 role flows complete without manual workarounds on a real device ✓ No console errors during a full walkthrough ✓ Empty states (no orders, no products) show friendly messages
M1-14	**MOB-1**	**Support, Referral & Notification Screens**	• Build “Write to us” / feedback screens with star rating + thank-you confirmation • Build “Refer a friend” screen with a shareable code • Build notification preferences, account settings, and T&C screens	✓ Feedback submits to POST /support/tickets and shows a confirmation ✓ Referral code is shareable via the native share sheet and trackable ✓ Notification toggle changes persist via the API
M1-15	**MOB-1**	**EAS Build + APK Release**	• Optimize bundle: npx expo export; target APK size <20MB • Configure eas.json production profile + Android keystore • Run eas build --platform android --profile production • Install APK on a test device, final smoke test across all screens	✓ APK installs and runs on Android 10+ without crashes ✓ EAS Build completes successfully ✓ All 3 role flows tested on the production APK

Appendix · Module Ownership Matrix

MODULE	OWNER	MOBILE OWNER	CRITICAL DEPENDENCY
Auth (Firebase + Email OTP + JWT)	BE-1	MOB-1	RS256 key pair in Railway env
Shop Onboarding + KYC	BE-1	MOB-1	Cloudinary signed upload preset
Catalog + Category CRUD	BE-2	MOB-1	DB Schema Part 2 (B2-01)
Search (pg_trgm)	BE-2	MOB-1	GIN index on Product.name (B2-01)
Cart	BE-2	MOB-1	Catalog module complete
Order State Machine	BE-3	MOB-1	Product + inventory module complete
Payments + Wallet	BE-3	MOB-1	Razorpay test-mode keys (B3-03)
Delivery + Geofenced Tracking	BE-2	MOB-1	Order FSM complete (B3-06)
Notifications (FCM)	BE-1	MOB-1	Firebase service account (B1-05)
Admin Ops + Audit	BE-1	MOB-1	All modules partially complete (W3)
CI/CD + Infrastructure	BE-1	—	GitHub Secrets + Railway setup

49 tasks · 4 weeks · 4 developers — Storefiller Team Execution Handbook.